REVISITING LIFETIME-AWARE VM ALLOCATION: RECONSTRUCTION, DIAGNOSIS AND PRIOR-FUSED LIFETIME PREDICTION

CE7490 Course Project

CE7490 Course Project Report · Revised 29 October 2026

ABSTRACT

Lifetime-aware VM allocation uses predicted departures to reduce fragmentation and increase the fraction of empty hosts. This project reconstructs an event-driven CPU and memory simulator of LAVA [1] from a surviving course report after the original code and traces became unavailable, and studies how lifetime-prediction quality and uncertainty handling change the scheduling benefit. With 180 hosts and 2,500 VMs per paired synthetic trace, LAVA's gain over Best-Fit grows with controlled prediction accuracy, but at 25% accuracy LAVA is significantly worse than Best-Fit (−0.187 [−0.240, −0.135] percentage points, pp, on 100 fresh seeds).

The earlier report proposed Host-Level Confidence-Aware LAVA as the remedy. It does not outperform Best-Fit on the report's held-out seeds and is significantly worse on 100 fresh seeds (−0.090 [−0.147, −0.034] pp). We present this as a diagnostic negative result. Three measured properties contribute: the default predictor's class confidence is inverted relative to predictability, the confidence gate passes essentially every request under controlled noise, and isolating hosts by mode costs about one percentage point of empty hosts during the arrival period.

We then propose **prior-fused lifetime prediction**: a Bayesian posterior that combines each noisy prediction with the profile-level lifetime distribution, treats the 14-day cap as censoring and conditions on uptime, while every scheduling rule stays unchanged. In a pre-registered, Holm-corrected evaluation on 100 fresh seeds, confidence-aware LAVA with the fused predictor beats LAVA in 3 of 4 registered tests (+0.224, +0.147 and +0.088 pp at 25%, 50% and 85% accuracy) and Best-Fit in 3 of 4; 6 of 8 hypotheses hold. A post-hoc decomposition suggests that most of the gain comes from the fused estimate rather than the gate. Under milder errors, misspecified error spreads, doubled prior means and doubled load, the gain over LAVA shrinks or grows and is not always significant, but it is never significantly negative except with halved prior means at 85% accuracy.

Principal findings

- **RQ1 – prediction quality is first-order.** On fresh seeds LAVA moves from −0.187 pp at 25% accuracy to +0.507 pp at 100%; the Oracle reference is +0.552 pp.
- **RQ2 – confidence routing is a diagnostic negative result.** Neither VM-level nor host-level routing beats Best-Fit under the default predictor; inverted confidence, a blind gate and the packing cost of isolation contribute.
- **RQ3 – prior fusion fixes the low-accuracy failure.** 3 of 4 registered tests against LAVA hold. Uncertainty helped inside the lifetime estimate, not as a policy switch. A prior that underestimates lifetimes is the main risk.

Keywords: VM scheduling, lifetime prediction, Bayesian fusion, censoring, uncertainty, paired experiments, pre-registration, reconstruction

1 INTRODUCTION AND RESEARCH SCOPE

A VM scheduler must place requests that consume several resources at once. A feasible placement can still leave CPU and memory in incompatible proportions and strand capacity. Departure times add a temporal dimension: VMs that fit equally well at arrival may keep a host occupied for very different periods. Empty hosts matter for maintenance, large requests and energy management, although this simulator measures empty capacity rather than those outcomes directly.

LAVA [1] combines learned lifetime distributions, re-prediction with uptime and host lifecycle adaptation. The production system relies on private traces and integration with Borg [2]. This project studies the public scheduling ideas in a transparent synthetic environment, where policies can be compared on identical workloads and identical prediction realizations.

Related work

Resource Central [3] showed that VM lifetimes in a large cloud are predictable from history and can inform resource management. Barbalho et al. [4] use lifetime predictions to align departures when packing VMs; our LA-Binary baseline follows that idea as evaluated in [1]. Protean [5] describes the production allocation service into which such policies plug. LAVA [1] adds lifetime distributions, re-prediction and host lifecycle states. Our prior-fused predictor applies standard Bayesian updating and the treatment of right-censored observations from survival analysis [6] to the noisy predictions that LAVA consumes.

Research questions

RQ1. How does controlled lifetime-prediction accuracy change LAVA's improvement over a resource-only Best-Fit baseline?

RQ2. Does routing low-confidence requests to Best-Fit, per VM or per host, make LAVA robust to unreliable predictions?

RQ3. Can the lifetime estimate itself be made robust by fusing each noisy prediction with the profile-level lifetime distribution, without changing the scheduler?

What changed in this revision

The surviving report [7] claimed that Host-Level CA significantly improved the default synthetic regime. The previous revision replaced those values with reconstructed measurements and found no such gain. This revision makes three changes. It turns that outcome into a **diagnostic negative result** (Section 7). It introduces **prior-fused prediction** as the new main contribution (Section 8). It evaluates the method under a **pre-registered protocol** on previously unused seeds, with robustness variants (Sections 9 and 10).

After the change, the six original policies reproduce all 1,644 stored runs field by field on macOS (Section 5). The only historical value from the surviving report appears, labeled, in Section 7.1.

Contributions

- A runnable reconstruction of Best-Fit, LA-Binary, NILAS, LAVA and two confidence-aware variants, with paired statistics, resource assertions and raw per-seed results.
- A diagnosis of why confidence-aware routing does not help in this setting, supported by profile-level confidence analysis, gate pass rates and a time-window decomposition.
- Prior-fused lifetime prediction: a drop-in Bayesian predictor, with censoring at the prediction cap and survival-consistent re-prediction, that leaves every scheduling rule unchanged.
- A pre-registered, Holm-corrected evaluation on 100 fresh seeds, plus robustness to error severity, model misspecification and load (23,240 additional simulations).

2 SCHEDULING ALGORITHMS

Table 2.1. Policies compared. "*-Bayes" denotes the four variants that only swap in the prior-fused predictor.

Policy	Placement signal	Prediction use
Best-Fit	Minimize squared residual CPU plus squared residual memory	None for placement
LA-Binary	Prefer non-empty hosts with the same initial short/long label, then other non-empty hosts, then empty hosts; Best-Fit within each group	One-shot label at 2 hours
NILAS	Quantized extension of the predicted latest host exit, then Best-Fit	Re-predicts live residents
LAVA	Lifecycle state and lifetime class, then NILAS and Best-Fit	Re-predicted remaining lifetime
VM-Level CA (VM-CA)	LAVA or Best-Fit, chosen per arrival	Class confidence $\geq$ threshold τ
Host-Level CA (Host-CA)	Same gate; eligible hosts restricted to the same mode or empty	Same gate as VM-CA
*-Bayes variants (new)	Identical rules to NILAS, LAVA, VM-CA or Host-CA	Prior-fused prediction (Section 8)

Host lifecycle and placement order

LAVA classes use remaining-lifetime boundaries at 1, 10 and 100 hours, supplemented from the public paper [1]. A newly occupied host is *open*. When CPU or memory reaches 90%, it becomes *recycling* and records its current VMs as the residual cohort. Placement prefers recycling hosts in a higher class than the arriving VM (closest class first), then open hosts in the same class, then other non-empty hosts and finally empty hosts. When the residual cohort exits, a non-empty recycling host drops one class; deadline expiry raises the class and refreshes the cohort.

Confidence-aware routing

VM-CA routes each arrival by its confidence. Because hosts are shared, one host can receive both LAVA and Best-Fit decisions; we call this *mode mixing*. Host-CA assigns a mode at a host's first placement and keeps it until the host empties. It considers same-mode and empty hosts first and allows a cross-mode placement only when no such host can admit the request (capacity fallback). No fallback occurred in any recorded suite.

Prior-fused variants

The four *-Bayes policies run exactly the scheduling code of their base policy. Only the predictor object differs, so any difference in outcome is attributable to the lifetime estimate and its confidence. Under the default predictor and the Oracle there is no noisy value to fuse, and the variants reproduce their base policies bit for bit.

3 Workload and parameter provenance

Each host has normalized CPU and memory capacity of one. Every trace contains 2,500 VM requests at a documented mean arrival rate of eight per simulated hour. The reconstruction uses independent exponential interarrival times; the surviving report gives the mean rate but not the arrival process, so Poisson arrivals are an explicit assumption (Table 3.1).

Table 3.1. Simulator parameters and their provenance.

Parameter	Value	Status
Hosts / requests	180 / 2,500 per trace	Recovered
Mean arrival rate	8 requests/hour	Recovered
Interarrival distribution	Independent exponential	Inferred
Profile probabilities	Interactive 0.45, batch 0.35, service 0.20	Inferred
Prediction cap	336 hours (14 days)	Recovered
Recycling threshold	0.90 CPU or memory	Recovered
LAVA class boundaries	1, 10, 100 hours	Public paper supplement
Best-Fit score	Residual CPU squared + residual memory squared	Inferred

Two-exponential lifetime mixtures

For each profile, p is the probability of the short component and a, b are the component means (Table 3.2). The profile names and the two-component family are recovered; all numeric values are declared synthetic defaults.

Table 3.2. Lifetime mixture of each VM profile.

Profile	p (short component)	a (hours)	b (hours)
Interactive	0.9	0.3	8
Batch	0.65	2	30
Service	0.3	4	120

CPU and memory shapes

The inferred (CPU, memory) shapes are (0.125, 0.125), (0.25, 0.125), (0.125, 0.25), (0.5, 0.25) and (0.25, 0.5) with probabilities 0.35, 0.20, 0.20, 0.125 and 0.125. Shape, profile and component are sampled independently. Lifetimes are not truncated; only predictions receive the 14-day cap.

These values were declared once and never fitted to the earlier report or to any result. A different plausible mixture could change relative policy performance; Section 10 varies predictor-side assumptions and the load, but not the generating mixture itself.

4 Lifetime prediction and confidence

Conditional remaining lifetime

Let T follow a two-exponential mixture with short-component probability p and means a and b. At uptime u the short-component weight and the expected remaining lifetime are

$$w(u) = p\cdot e^{-u/a} / [\, p\cdot e^{-u/a} + (1-p)\cdot e^{-u/b} \,],$$

$$E[T - u \mid T > u] = w(u)\cdot a + (1 - w(u))\cdot b.$$

The weights are evaluated in log space to avoid underflow. The *default predictor* returns this conditional mean, capped at 336 hours. Its confidence is the conditional probability that the remaining lifetime falls in the LAVA class that contains the mean.

Controlled prediction accuracy

Following the controlled-noise protocol of [1, Appendix G.1], the harness assigns each VM to the *correct bucket* with probability equal to the requested accuracy and to the *incorrect bucket* otherwise. It applies Gaussian noise in log10 space with $\sigma = 0.001$ (correct) or $\sigma = 3$ (incorrect) to the true total lifetime and caps the result at 14 days. All policies with the same seed and setting see the same noisy values; the bucket label never reaches the scheduler.

The original controlled predictor freezes the noisy value. Before that value is reached, remaining lifetime is the

value minus uptime; afterwards it switches to min(336, max(0.1, uptime)) hours. Its confidence is a posterior class mass under a log-uniform prior and the global error model (accuracy and the two σ values).

Strict Oracle

Oracle predictions use the actual remaining lifetime with confidence one. This full-information reference measures how much the same scheduling rules can gain with perfect lifetimes. It is not an upper bound for every policy (Section 11).

Three properties that matter later

- The default confidence is the probability of the class that contains the mixture *mean*. For bimodal mixtures the mean often lies in a low-probability class (Section 7.3).
- The controlled predictor trusts every noisy value, including errors that span several orders of magnitude (Section 7.4).
- When a VM outlives its noisy value, its predicted remaining lifetime jumps to min(336, max(0.1, uptime)), and a value at the 336-hour cap is treated as an exact observation rather than as censored.

Section 7 shows how the first two properties affect confidence-aware routing. Section 8 addresses the second and third directly.

5 Experimental protocol and verification

Table 5.1. Experiment suites. Controlled sweep accuracies are 25%, 50%, 75%, 80%, 85%, 90%, 95% and 100%; later suites use 25%, 50%, 75%, 85% and 100%. *Added after review; descriptive.

Suite	Seeds	Settings and policies	Runs
Smoke	0–1	3 predictor settings, 6 policies	36
Initial sweep	0–19	8 accuracies + default + Oracle; Best-Fit, LAVA, VM-CA	600
Validation	200–205	Default predictor; 7 thresholds; 4 policies	168
Held-out	20–39	5 accuracies + default + Oracle; 6 policies	840
Improvement test	1000–1099	5 accuracies + default + Oracle; 10 policies (pre-registered)	7,000
Legacy rerun	20–39	Same 7 settings; 10 policies	1,400
Threshold sensitivity	200–219	4 accuracies; 7 thresholds; 4 policies	2,240
Robustness	2000–2049	4 accuracies; 7 predictor or load variants; 7 policies	9,800
Robustness control*	2000–2049	4 accuracies; unmodified protocol; 7 policies	1,400
Load ×2*	2000–2049	4 accuracies; arrival rate 16/h, 5,000 VMs; 7 policies	1,400

Metrics

The primary metric is 100 times the time integral of the number of empty hosts, divided by the number of hosts and the interval from the first arrival to the final exit. It includes the full drain tail. Results are paired differences in percentage points (pp) relative to the same-seed baseline. A secondary diagnostic integrates only the arrival window (first to last arrival) to separate packing during the busy period from consolidation during the drain tail.

Scale matters for reading the results. Best-Fit leaves 89.6% of host-time empty on average (86.3–93.4% across test seeds), because the offered load is low and the drain tail is long. The arrival window lasts about 312 hours of a 985-hour horizon, so about 68% of the primary metric is the drain tail. With 180 hosts, 1 pp equals 1.8 hosts on average over the horizon. The headline gain of +0.224 pp is therefore about 0.40 of a host, or 398 empty host-hours per trace (2.2% of Best-Fit's busy host-hours).

The offered load is about 20% at steady state (8 VMs per hour × 21.65 h mean lifetime × 0.2125 CPU, over 180 hosts). Because the 2,500 arrivals end before steady state is fully reached, realized CPU utilization is about 14% during the arrival window and 7% over the full horizon.

Inference

For each seed the analysis subtracts the baseline metric from the alternative and forms a two-sided Student-t 95% interval with n−1 degrees of freedom. It refuses comparisons with missing pairs, failed admissions or unequal accepted counts. Intervals in all tables are unadjusted. Only the eight registered tests of Section 9 also carry Holm–Bonferroni adjusted p-values [8]; all other comparisons are descriptive.

Pre-registration

Development of the new method used only seeds 0–19. The hypotheses, test seeds, frozen threshold, correction and decision rule were written to *configs/improvement_protocol.json*, with SHA256 hashes

of all simulator sources, at 20:10 (UTC+8) on 28 October 2026. This was before any prior-fused run on the legacy, threshold, test or robustness seeds. The registration is a local file with a self-reported timestamp, not an external registry. No simulator source changed afterwards, and all six improvement-suite manifests record the registered hashes. The development exploration script was packaged after registration; it reproduces exactly the development numbers recorded in the registration file. The two suites marked * in Table 5.1 were added after review.

Verification

All 24,884 runs accept every request with zero scheduling failures and no oversubscription. Twenty-nine unit tests pass, covering event ordering, time integration, mixture conditioning, resource bounds, host isolation, capacity fallback, Oracle equivalence, the no-truth-access boundary of both predictors, censoring, and the paired t-test and Holm adjustment. Re-simulating the 1,644 stored reconstruction runs reproduces every field bit for bit on macOS; on Linux all integer decision counts match and floating-point values differ only in the last digits (relative difference $\leq 3.3\times10^{-16}$). In the other direction, an evenly spaced sample of 300 improvement runs computed on the Linux server, re-simulated on macOS, matches every integer field, with floating-point differences of at most 5.2×10^{-16}. *scripts/verify_reproduction.py* performs these checks and writes *results/verification_log.txt*.

Compute

The simulator is pure-Python CPU code and needs no GPU. A report-scale run takes about 0.1–0.3 s for the original policies and 0.7 s with the fused predictor on one Apple M5 core, and about 0.9 s on average on one core of the server used below. The 23,240 improvement runs used 64 low-priority processes on a 192-thread Xeon server and finished in about 6 minutes of wall time; the server's H20 GPUs were not used.

6 RQ1: Prediction accuracy sensitivity

Figure 6.1 and Table 6.1 show LAVA minus Best-Fit across controlled accuracy on the initial split.

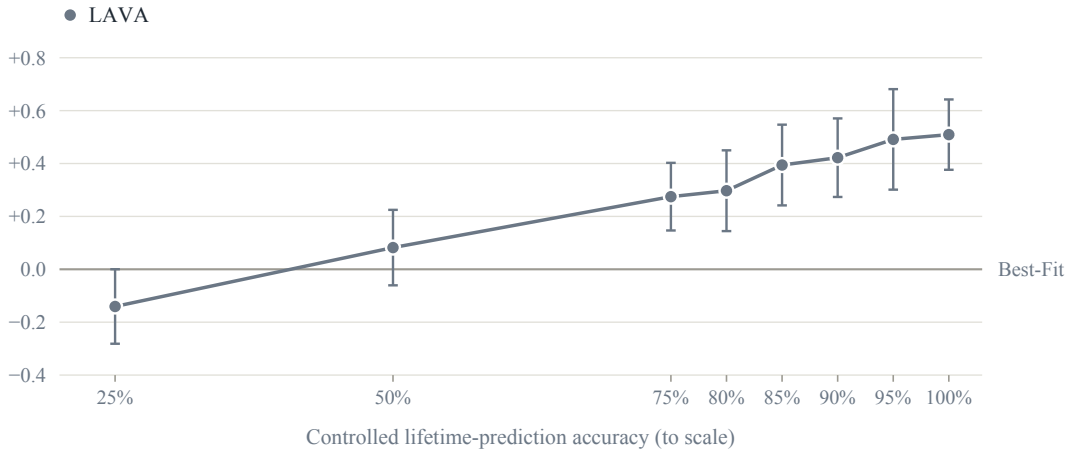


Figure 6.1. LAVA minus Best-Fit across controlled accuracy, initial split (seeds 0–19). Percentage points; paired means with 95% intervals, n = 20 per point; the accuracy axis is to scale.

Table 6.1. Values plotted in Figure 6.1.

Accuracy / predictor	LAVA − Best-Fit, mean [95% CI], pp
25%	−0.141 [−0.282, 0.000]
50%	+0.082 [−0.061, +0.225]
75%	+0.275 [+0.147, +0.403]
80%	+0.297 [+0.144, +0.450]
85%	+0.394 [+0.242, +0.547]
90%	+0.422 [+0.274, +0.571]
95%	+0.491 [+0.301, +0.681]
100%	+0.509 [+0.376, +0.642]
Default predictor	+0.124 [+0.005, +0.242]
Oracle	+0.575 [+0.433, +0.718]

The first tested level with a positive paired interval is 75%, and the estimated gain keeps increasing through the high-accuracy regime. This answers RQ1 within the reconstructed workload. The significance point is a property of the tested grid, the workload and the implementation, not a universal threshold; the scan is descriptive and not corrected for multiplicity.

The 100 fresh test seeds of Section 9 confirm the shape with tighter intervals: LAVA minus Best-Fit is −0.187 [−0.240, −0.135] pp at 25%, −0.025 [−0.077, +0.027] at 50%, +0.218 [+0.162, +0.274] at 75%, +0.309 [+0.250, +0.369] at 85% and +0.507 [+0.445, +0.569] at 100%; the Oracle reaches +0.552 [+0.491, +0.613]. **At 25% accuracy LAVA is significantly worse than Best-Fit**: trusting unreliable predictions harms packing. This is the regime that RQ2 and RQ3 target.

Under the default predictor, LAVA's small gain on the initial split (+0.124 [+0.005, +0.242] pp) does not replicate on the held-out seeds (+0.001 [−0.104, +0.105]) or on the fresh seeds (−0.043 [−0.098, +0.012]).

7 RQ2: CONFIDENCE-AWARE ROUTING, A DIAGNOSTIC NEGATIVE RESULT

7.1 Held-out and fresh-seed outcome

Table 7.1 reports the held-out comparison (seeds 20–39) relative to Best-Fit, with the threshold fixed at the report value 0.3. At 50% accuracy and above, both confidence-aware variants follow LAVA exactly, because the gate passes every arrival.

Table 7.1. Held-out seeds 20–39, pp versus Best-Fit, mean [95% CI].

Setting	LAVA	VM-CA	Host-CA
25%	−0.137 [−0.247, −0.026]	−0.145 [−0.261, −0.030]	−0.115 [−0.234, +0.004]
50%	+0.069 [−0.022, +0.159]	+0.069 [−0.022, +0.159]	+0.069 [−0.022, +0.159]
75%	+0.215 [+0.096, +0.333]	+0.215 [+0.096, +0.333]	+0.215 [+0.096, +0.333]
85%	+0.399 [+0.281, +0.517]	+0.399 [+0.281, +0.517]	+0.399 [+0.281, +0.517]
100%	+0.598 [+0.490, +0.706]	+0.598 [+0.490, +0.706]	+0.598 [+0.490, +0.706]
Synthetic	+0.001 [−0.104, +0.105]	+0.114 [+0.007, +0.220]	−0.045 [−0.138, +0.049]
Oracle	+0.661 [+0.549, +0.774]	+0.661 [+0.549, +0.774]	+0.661 [+0.549, +0.774]

Under the default predictor, Host-CA does not improve on Best-Fit on the held-out seeds and is significantly worse than VM-CA there (Table 7.2). On 100 fresh seeds it is significantly worse than Best-Fit (−0.090 [−0.147, −0.034] pp). The small held-out gain of VM-CA does not replicate (−0.015 [−0.070, +0.041] pp). For reference, the surviving report gave Host-CA minus Best-Fit as +0.241 [+0.125, +0.357] pp; this historical value is not reproduced.

Table 7.2. Paired comparisons of Host-CA under the default predictor.

Default predictor, Host-CA minus …	Held-out seeds 20–39 (n = 20)	Fresh seeds 1000–1099 (n = 100)
Best-Fit	−0.045 [−0.138, +0.049]	−0.090 [−0.147, −0.034]
LAVA	−0.046 [−0.169, +0.078]	−0.047 [−0.097, +0.002]
VM-CA	−0.158 [−0.303, −0.013]	−0.075 [−0.130, −0.021]

7.2 The mechanism works as designed

Host isolation removes exactly what it was designed to remove. Both variants receive the same gate sequence, and Host-CA reduces both cross-mode counts to zero without any capacity fallback (Table 7.3). The missing gain is therefore not an implementation defect of the invariant.

Table 7.3. Mechanism counts under the default predictor, held-out seeds.

Diagnostic, mean per trace (seeds 20–39)	VM-CA	Host-CA
Gate changes per 1,000 arrivals	318.64	318.64
Best-Fit decisions placed on LAVA-mode hosts	607.75	0
LAVA decisions placed on Best-Fit-mode hosts	319.40	0
Capacity fallbacks	0	0

7.3 Diagnosis 1: confidence is inverted relative to predictability

Under the default predictor every request arrives with uptime zero, so its confidence is a fixed number per profile (Table 7.4, Figure 7.1). The confidence measures the probability of the class that contains the mixture mean, which for bimodal mixtures is often an unlikely class. Interactive VMs are the most predictable profile, yet they receive the lowest confidence; service VMs are the least predictable, yet they receive the highest. At $\tau = 0.3$ only service VMs pass the gate, so Host-CA reduces to isolating service VMs on their own hosts.

Table 7.4. Class probabilities at arrival (most likely class in bold) and the confidence used by the gate.

Profile	P(<1 h)	P(1–10 h)	P(10–100 h)	P(≥100 h)	Mean	Class of mean	Gate confidence
Interactive	**0.880**	0.092	0.029	0.000	1.07 h	1–10 h	0.092
Batch	0.267	**0.478**	0.243	0.012	11.80 h	10–100 h	0.243
Service	0.072	0.259	**0.364**	0.304	85.20 h	10–100 h	0.364

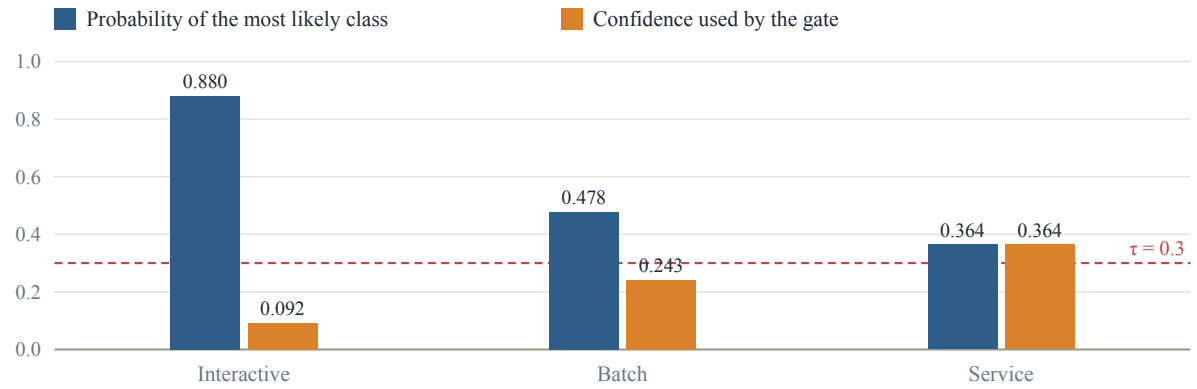


Figure 7.1. Predictability versus gate confidence per profile. The gate confidence is lowest for the most predictable profile and highest for the least predictable one.

Fixing the inversion alone is not enough. On the development seeds, routing by the most likely class and its probability did not beat LAVA at any tested threshold (−0.129 to −0.043 pp versus LAVA for the VM-level and host-level variants at $\tau = 0.4$, 0.5 and 0.9; Section 8). The inversion is a contributing factor, not a sufficient explanation.

7.4 Diagnosis 2: the gate is blind under controlled noise

At arrival, the correct-bucket error of the original controlled predictor lies almost entirely inside the predicted class, so the confidence is at least about the global accuracy. At 50% accuracy and above this bound alone exceeds $\tau = 0.3$; at 25% almost every request still passes (Table 7.5). Confidence-aware routing therefore cannot separate the errors that actually harm packing.

Table 7.5. Share of arrivals routed to LAVA at $\tau = 0.3$, fresh seeds (n = 100); fused confidence (Section 8) for comparison.

Accuracy	VM-CA (original confidence)	VM-CA-Bayes (fused confidence)
25%	99.96%	75.46%
50%	100.00%	78.74%
75%	100.00%	80.58%
85%	100.00%	84.77%
100%	100.00%	100.00%
Synthetic	20.09%	20.09%

7.5 Diagnosis 3: isolation costs packing

Host-CA restricts every request to hosts of its own mode or to empty hosts, so requests often start new hosts that could otherwise have been shared. During the arrival window Host-CA has 0.99 pp fewer empty hosts than Best-Fit; over the full horizon the deficit is 0.09 pp, so the drain tail recovers about 72% of the lost empty host-hours (Table 7.6). Combining isolation with the fused predictor makes it worse (Section 9).

Table 7.6. Arrival-window and full-horizon differences, fresh seeds (n = 100), pp.

Comparison	Setting	Arrival window	Full horizon (primary)
Host-CA − Best-Fit	Synthetic	−0.992 [−1.028, −0.957]	−0.090 [−0.147, −0.034]
Host-CA − VM-CA	Synthetic	−0.857 [−0.894, −0.821]	−0.075 [−0.130, −0.021]
Host-CA-Bayes − VM-CA-Bayes	50%	−0.699 [−0.734, −0.664]	−0.284 [−0.342, −0.226]

7.6 Threshold validation

On validation seeds 200–205 the reconstructed objective (mean Host-CA improvement over LAVA and VM-CA) selects $\tau = 0.5$ by smallest-threshold tie-breaking: $\tau = 0.5$, 0.7, 0.8, 0.9 tie at −0.085 pp, because at these thresholds no profile passes the gate and Host-CA equals Best-Fit. Every candidate score is negative, so changing the threshold cannot rescue host-level routing; the held-out comparisons keep the frozen report value 0.3.

Conclusion for RQ2

Confidence-aware routing does not help here, and threshold tuning did not change that. The confidence signal is inverted under the default predictor and uninformative under controlled noise, and host isolation has a measurable packing cost. Mode mixing is real, but removing it does not produce a gain. The negative result points at the predictor, which RQ3 changes.

8 RQ3: PRIOR-FUSED LIFETIME PREDICTION

The diagnosis suggests repairing the lifetime estimate rather than switching policies. The scheduler already knows a profile-level lifetime distribution, because the default predictor is exactly that model. Under controlled noise it also receives a per-VM noisy value whose error model is globally known. Prior fusion combines both sources with Bayes' rule.

Model

- **Prior.** The VM's profile mixture, the same two-exponential model used by the default predictor.
- **Likelihood.** $\log10\ o = \log10\ T + \varepsilon$ with $\varepsilon \sim \text{acc}\cdot N(0, 0.001^2) + (1-\text{acc})\cdot N(0, 3^2)$. The correct-bucket component is far narrower than any grid, so its weight is the global accuracy times the prior density of $\log10\ T$ at $\log10\ o$. It is represented by nine atoms one σ apart over $\pm 4\sigma$.
- **Censoring.** A value at the 336-hour cap contributes $P(\log10\ T + \varepsilon \geq \log10\ 336)$, not a point likelihood [6].
- **Posterior.** 425 weighted atoms on a log10 grid from 10^{-4} to $10^{4.5}$ hours (50 per decade), plus the correct-bucket atoms. Suffix sums make each re-prediction O(log n).
- **Uptime conditioning.** Keep the atoms above the current age. An overdue VM therefore falls back smoothly to the incorrect-bucket component, which removes the jump to max(0.1, uptime). If no atom remains, the predictor falls back to the profile prediction.
- **Outputs.** The posterior mean remaining lifetime (capped at 336 hours) and the posterior mass of its LAVA class. This is the same interface as the original predictor, so the NILAS, LAVA and CA rules are unchanged.

Information boundary

The fused predictor uses the noisy value, the global accuracy, the error spreads, the cap and the profile. These are the quantities the original controlled confidence uses, plus the profile model that the default predictor already uses. That prior is the generating distribution; Section 10 tests misspecified priors. The predictor never sees the bucket label or the true lifetime. A unit test rebuilds it on records whose true durations are multiplied by 50, keeps the same noisy values and verifies identical predictions.

Worked examples

- Interactive VM, accuracy 25%, noisy value 300 h. The original predictor returns 300 h. The posterior mean is 1.52 h, close to the profile mean of 1.07 h, because 300 h is implausible under the interactive prior.
- Service VM, accuracy 50%, noisy value 2 h. A plausible value moves the posterior mean to 27.0 h, against the prior mean of 85.2 h and the raw value of 2 h.
- Service VM, accuracy 100%, correct value at the 336-hour cap, re-predicted later. At uptime 300 h the original predictor expects 36 h remaining and the fused one 151 h, because the true lifetime is only known to exceed 336 h. At 400 h the original jumps to 336 h with confidence 0.0, while the fused one expects 118 h.

Development history

Two other ideas were explored on development seeds 0–19 and rejected before registration. An expected host-exit extension score computed from the full mixture gave no gain under the default predictor (LAVA variant −0.194 [−0.312, −0.076] pp versus LAVA). Routing by the most likely class and its probability did not beat LAVA at any tested threshold (−0.129 to −0.043 pp). On these seeds the confidence gate also appeared to add value

on top of fusion: VM-CA-Bayes minus LAVA was +0.316, +0.187 and +0.134 pp at 25%, 50% and 75%, against +0.249, +0.100 and +0.049 pp for LAVA-Bayes. This is why VM-CA-Bayes was registered as the primary variant; Section 9 shows the gate's advantage did not replicate. The exploration script and its outputs are part of the artifact (scripts/explore_dev.py).

9 Pre-registered evaluation of prior fusion

The registered family compares VM-CA-Bayes against LAVA and against Best-Fit at 25%, 50%, 75% and 85% accuracy on fresh seeds 1000–1099 (n = 100). VM-CA-Bayes was registered because the project studies confidence-aware scheduling and it was the strongest variant on development seeds; 100% accuracy was excluded because the two predictors then differ only in overdue handling. The threshold stays at the frozen report value 0.3, and no parameter was tuned on any seed. An improvement is claimed only if the mean is positive and the Holm-adjusted p-value is below 0.05.

Table 9.1. Registered family (8 tests). Intervals are unadjusted; p-values are Holm-adjusted. 3 of 4 tests against LAVA and 3 of 4 against Best-Fit hold.

Accuracy	VM-CA-Bayes − LAVA	Holm p	Holds	VM-CA-Bayes − Best-Fit	Holm p	Holds
25%	+0.224 [+0.174, +0.275]	<0.001	yes	+0.037 [−0.012, +0.086]	0.14	no
50%	+0.147 [+0.088, +0.205]	<0.001	yes	+0.122 [+0.064, +0.180]	<0.001	yes
75%	+0.054 [−0.001, +0.109]	0.11	no	+0.272 [+0.217, +0.327]	<0.001	yes
85%	+0.088 [+0.037, +0.139]	0.003	yes	+0.398 [+0.338, +0.457]	<0.001	yes

Against LAVA, three of four tests hold. At 75% the interval includes zero (lower bound −0.001; Holm p = 0.11). Against Best-Fit, the tests at 75% and 85% would also hold for unmodified LAVA (+0.218 and +0.309 pp), so the evidence specific to fusion is the three LAVA tests and the Best-Fit test at 50%. At 25% the registered test against Best-Fit fails (+0.037 [−0.012, +0.086] pp). Secondary, unadjusted comparisons at 25% put NILAS-Bayes (+0.084 [+0.035, +0.134], p < 0.001) and LAVA-Bayes (+0.055 [+0.006, +0.104], p = 0.03) above Best-Fit; they were not registered.

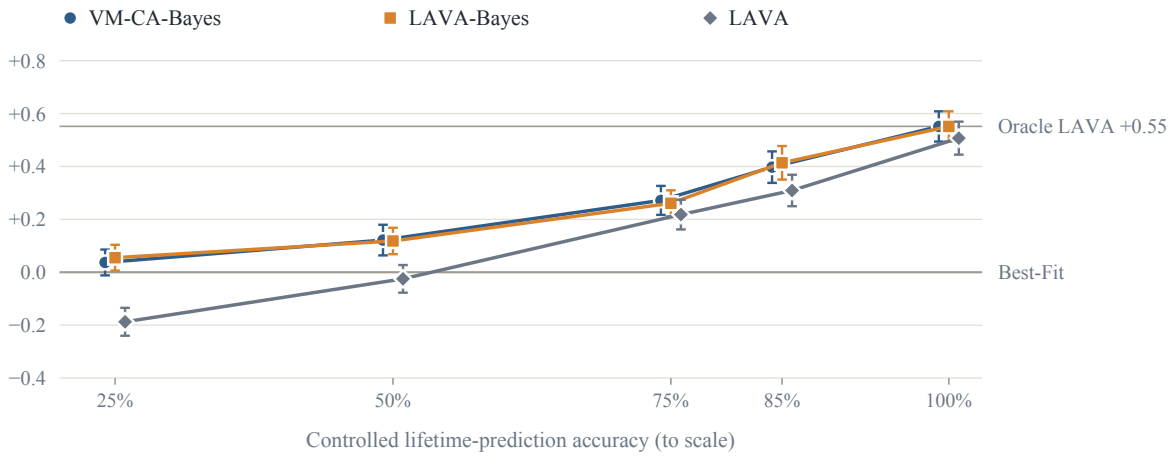


Figure 9.1. Gain over Best-Fit versus controlled accuracy, fresh seeds 1000–1099. VM-CA-Bayes is the registered variant; LAVA-Bayes (not registered) and LAVA are shown for reference. Paired means with unadjusted 95% intervals, n = 100; the accuracy axis is to scale.

All policies versus Best-Fit

Tables 9.2 and 9.3 list every policy on the test seeds. LA-Binary, omitted from the tables, is slightly below Best-Fit in every setting (−0.056 to −0.114 pp).

Table 9.2. Original policies, fresh seeds, pp versus Best-Fit. Shading marks intervals entirely above (green) or below (red) zero.

Setting	LAVA	NILAS	VM-CA	Host-CA
25%	−0.187 [−0.240, −0.135]	−0.055 [−0.108, −0.001]	−0.178 [−0.231, −0.125]	−0.180 [−0.236, −0.125]
50%	−0.025 [−0.077, +0.027]	+0.076 [+0.018, +0.134]	−0.025 [−0.077, +0.027]	−0.025 [−0.077, +0.027]
75%	+0.218 [+0.162, +0.274]	+0.218 [+0.158, +0.279]	+0.218 [+0.162, +0.274]	+0.218 [+0.162, +0.274]
85%	+0.309 [+0.250, +0.369]	+0.349 [+0.294, +0.403]	+0.309 [+0.250, +0.369]	+0.309 [+0.250, +0.369]
100%	+0.507 [+0.445, +0.569]	+0.476 [+0.420, +0.531]	+0.507 [+0.445, +0.569]	+0.507 [+0.445, +0.569]
Synthetic	−0.043 [−0.098, +0.012]	+0.049 [+0.002, +0.096]	−0.015 [−0.070, +0.041]	−0.090 [−0.147, −0.034]
Oracle	+0.552 [+0.491, +0.613]	+0.581 [+0.518, +0.644]	+0.552 [+0.491, +0.613]	+0.552 [+0.491, +0.613]

Table 9.3. Prior-fused variants on the same seeds; identical to their base policies under the default predictor and the Oracle.

Setting	NILAS-Bayes	LAVA-Bayes	VM-CA-Bayes	Host-CA-Bayes
25%	+0.084 [+0.035, +0.134]	+0.055 [+0.006, +0.104]	+0.037 [−0.012, +0.086]	−0.271 [−0.323, −0.220]
50%	+0.135 [+0.084, +0.187]	+0.118 [+0.068, +0.168]	+0.122 [+0.064, +0.180]	−0.162 [−0.221, −0.104]
75%	+0.324 [+0.268, +0.380]	+0.261 [+0.212, +0.310]	+0.272 [+0.217, +0.327]	+0.030 [−0.034, +0.093]
85%	+0.386 [+0.332, +0.441]	+0.414 [+0.350, +0.477]	+0.398 [+0.338, +0.457]	+0.199 [+0.139, +0.258]
100%	+0.599 [+0.538, +0.659]	+0.551 [+0.494, +0.609]	+0.551 [+0.494, +0.609]	+0.551 [+0.494, +0.609]
Synthetic	+0.049 [+0.002, +0.096]	−0.043 [−0.098, +0.012]	−0.015 [−0.070, +0.041]	−0.090 [−0.147, −0.034]
Oracle	+0.581 [+0.518, +0.644]	+0.552 [+0.491, +0.613]	+0.552 [+0.491, +0.613]	+0.552 [+0.491, +0.613]

Only the predictor changes

Table 9.4 compares each fused variant with its own base policy, so the scheduling rules are identical within each column. Fusion helps NILAS, LAVA and VM-CA at most accuracies, most where predictions are least reliable. It makes Host-CA significantly worse at every accuracy from 25% to 85% (−0.091 to −0.189 pp), consistent with the more selective gate splitting the fleet further (Diagnosis 3).

Table 9.4. Fused variant minus its base policy, fresh seeds, pp.

Accuracy	NILAS-Bayes − NILAS	LAVA-Bayes − LAVA	VM-CA-Bayes − VM-CA	Host-CA-Bayes − Host-CA
25%	+0.139 [+0.094, +0.185]	+0.242 [+0.182, +0.303]	+0.215 [+0.164, +0.266]	−0.091 [−0.146, −0.036]
50%	+0.059 [+0.005, +0.113]	+0.143 [+0.090, +0.197]	+0.147 [+0.088, +0.205]	−0.137 [−0.190, −0.085]
75%	+0.106 [+0.057, +0.154]	+0.043 [−0.010, +0.095]	+0.054 [−0.001, +0.109]	−0.189 [−0.246, −0.131]
85%	+0.038 [−0.012, +0.088]	+0.105 [+0.054, +0.155]	+0.088 [+0.037, +0.139]	−0.111 [−0.161, −0.061]
100%	+0.123 [+0.082, +0.164]	+0.044 [−0.005, +0.093]	+0.044 [−0.005, +0.093]	+0.044 [−0.005, +0.093]

Where the gain comes from (post hoc)

The comparisons in Tables 9.5 and 9.6 were not registered and are descriptive. Fusion improves on both of its ingredients (Table 9.5): the prior alone at every accuracy (second column, the same seeds under the default predictor) and the noisy predictions alone at every accuracy except 75%, where the interval includes zero (first column). Adding the confidence gate on top changes little: VM-CA-Bayes minus LAVA-Bayes lies between −0.018 and +0.011 pp, with intervals about ±0.05 pp wide, so a gate effect of that size cannot be excluded; the development-seed advantage of the gate did not replicate.

Table 9.5. Decomposition of the gain, fresh seeds, pp. The second column pairs each seed under controlled noise with the same seed under the default predictor; Best-Fit is identical in both.

Accuracy	LAVA-Bayes − LAVA (vs noisy predictions alone)	LAVA-Bayes − LAVA, default predictor (vs prior alone)	VM-CA-Bayes − LAVA-Bayes (gate on top)
25%	+0.242 [+0.182, +0.303]	+0.098 [+0.047, +0.149]	−0.018 [−0.066, +0.030]
50%	+0.143 [+0.090, +0.197]	+0.161 [+0.104, +0.218]	+0.003 [−0.050, +0.057]
75%	+0.043 [−0.010, +0.095]	+0.304 [+0.244, +0.363]	+0.011 [−0.043, +0.065]
85%	+0.105 [+0.054, +0.155]	+0.457 [+0.398, +0.515]	−0.016 [−0.066, +0.033]

The fused gate is more selective than the original one but discriminates only weakly between good and bad predictions (Table 9.6). It routes 82–87% of correct-bucket predictions to LAVA and 73–75% of incorrect-bucket ones, while the original gate routes essentially all of both. The fused estimate, rather than the gate, appears to absorb most of the damage from wrong predictions. NILAS-Bayes is nominally the strongest fused variant at low accuracy; that comparison would need its own confirmatory test.

Table 9.6. Share of arrivals routed to LAVA at $\tau = 0.3$ by the hidden error bucket, fresh seeds (post hoc; the bucket is recovered from the harness's random stream and never seen by the scheduler).

Accuracy	Original gate: correct	Original gate: incorrect	Fused gate: correct	Fused gate: incorrect
25%	99.9%	100.0%	82.2%	73.2%
50%	100.0%	100.0%	82.9%	74.6%
75%	100.0%	100.0%	83.0%	73.1%
85%	100.0%	100.0%	86.7%	73.9%

10 Robustness and sensitivity

The robustness suites use fresh seeds 2000–2049 (n = 50). Each variant changes one assumption: the severity of incorrect predictions, a misspecified error spread in the fusion model, a misspecified prior or the load. The first row of Tables 10.1 and 10.2 is the unmodified protocol on the same seeds. These comparisons are descriptive and not corrected for multiplicity.

Table 10.1. VM-CA-Bayes minus LAVA, pp, mean and [95% CI]. Green: interval above zero; red: interval below zero.

Variant (n = 50)	25%	50%	75%	85%
Protocol, same seeds	+0.225 [+0.141, +0.309]	+0.185 [+0.135, +0.235]	+0.113 [+0.043, +0.182]	+0.127 [+0.046, +0.208]
Incorrect-bucket σ = 1	+0.024 [−0.045, +0.093]	+0.102 [+0.013, +0.191]	+0.025 [−0.039, +0.089]	+0.093 [+0.025, +0.162]
Incorrect-bucket σ = 0.5	+0.070 [−0.009, +0.149]	+0.062 [−0.011, +0.135]	+0.037 [−0.037, +0.112]	+0.092 [+0.026, +0.159]
Fusion assumes σ = 3, true σ = 1	+0.083 [+0.008, +0.157]	+0.074 [+0.003, +0.146]	+0.002 [−0.073, +0.077]	+0.066 [0.000, +0.133]
Fusion assumes σ = 1, true σ = 3	+0.168 [+0.101, +0.236]	+0.173 [+0.116, +0.231]	+0.109 [+0.020, +0.199]	+0.069 [−0.011, +0.148]
Prior means × 0.5	+0.267 [+0.171, +0.362]	+0.135 [+0.061, +0.210]	−0.033 [−0.111, +0.045]	−0.116 [−0.197, −0.034]
Prior means × 2	+0.223 [+0.142, +0.303]	+0.215 [+0.135, +0.295]	+0.149 [+0.069, +0.229]	+0.112 [+0.043, +0.181]
Arrival rate × 2 (load × 2)	+0.476 [+0.374, +0.577]	+0.271 [+0.170, +0.372]	+0.177 [+0.088, +0.266]	+0.101 [−0.012, +0.215]

Table 10.2. VM-CA-Bayes minus Best-Fit under the same variants.

Variant (n = 50)	25%	50%	75%	85%
Protocol, same seeds	+0.039 [−0.037, +0.115]	+0.127 [+0.049, +0.204]	+0.280 [+0.204, +0.357]	+0.434 [+0.356, +0.513]
Incorrect-bucket σ = 1	+0.062 [−0.008, +0.131]	+0.196 [+0.119, +0.273]	+0.336 [+0.255, +0.417]	+0.464 [+0.385, +0.542]
Incorrect-bucket σ = 0.5	+0.189 [+0.111, +0.267]	+0.285 [+0.205, +0.365]	+0.389 [+0.315, +0.462]	+0.478 [+0.404, +0.553]
Fusion assumes σ = 3, true σ = 1	+0.120 [+0.047, +0.193]	+0.169 [+0.098, +0.239]	+0.313 [+0.241, +0.385]	+0.437 [+0.360, +0.513]
Fusion assumes σ = 1, true σ = 3	−0.017 [−0.086, +0.051]	+0.115 [+0.031, +0.199]	+0.277 [+0.193, +0.361]	+0.376 [+0.302, +0.450]
Prior means × 0.5	+0.081 [−0.015, +0.176]	+0.077 [−0.012, +0.166]	+0.135 [+0.065, +0.204]	+0.191 [+0.111, +0.272]
Prior means × 2	+0.037 [−0.044, +0.118]	+0.157 [+0.078, +0.236]	+0.317 [+0.262, +0.372]	+0.419 [+0.343, +0.495]
Arrival rate × 2 (load × 2)	+0.100 [+0.009, +0.192]	+0.262 [+0.145, +0.379]	+0.524 [+0.411, +0.637]	+0.666 [+0.544, +0.789]

Interpretation

- On the same seeds the unmodified protocol gives +0.225, +0.185, +0.113, +0.127 pp over LAVA at 25–85% accuracy, all with intervals above zero.
- Milder errors (σ = 1 or 0.5) make wrong predictions harder to recognize. Gains over LAVA shrink and are significant in only one or two cells, but never significantly negative; gains over Best-Fit remain significant from 50% accuracy upward.
- A misspecified error spread keeps a positive mean in every cell, with intervals above zero in two or three of four.
- Prior means × 2 keeps all four gains over LAVA. This is consistent with overestimation being the cheaper error here: a short VM on a long-lived host costs little, while a long VM on a short-lived host keeps it busy.
- **Failure mode.** With halved prior means, correct predictions of long lifetimes look implausible. For a service VM at 85% accuracy whose correct value sits at the 336-hour cap, the posterior mean drops from 251 h under the true prior to 72 h. VM-CA-Bayes is then worse than LAVA at 85% accuracy (−0.116 [−0.197, −0.034] pp; one descriptive cell of 32, unadjusted p = 0.006), while still better than Best-Fit (+0.191 [+0.111, +0.272] pp). A deployment needs a prior that does not underestimate lifetimes, or a learned prior.
- **Load × 2.** Doubling the arrival rate (16 VMs per hour, 5,000 VMs, same 180 hosts) lowers Best-Fit's empty share to 80.7%. LAVA's penalty at 25% accuracy deepens to −0.376 pp, and the fused advantage over LAVA grows at 25–75% accuracy (+0.476, +0.271, +0.177 pp against +0.225, +0.185, +0.113 on the protocol); at 85% it is +0.101 [−0.012, +0.215] pp.
- An earlier variant with 90 hosts turned out to be a pure rescaling: fewer than 90 hosts are ever busy and ties go to the lowest host index, so every placement equals the 180-host run and only the denominator of pp changes. It is not reported as a load test.

Threshold sensitivity

On threshold-sensitivity seeds 200–219 (Table 10.3), thresholds from 0.1 to 0.5 give a positive mean gain over LAVA at every accuracy, with intervals above zero in 12 of 16 cells. Thresholds of 0.7 and above still help at 25% accuracy but give negative means at 50–85%, with intervals below zero in 3 of 9 cells. These thresholds still route 64–80% of arrivals to LAVA at 75–85% accuracy, against 81–85% at τ = 0.3; sending the additional requests to Best-Fit costs more than it saves. The frozen τ = 0.3 was not chosen from these numbers.

Table 10.3. VM-CA-Bayes minus LAVA by threshold, seeds 200–219 (n = 20), pp.

τ	25%	50%	75%	85%
0.1	+0.208 [+0.079, +0.338]	+0.139 [+0.023, +0.256]	+0.109 [−0.024, +0.241]	+0.139 [+0.021, +0.258]
0.2	+0.268 [+0.185, +0.352]	+0.112 [+0.002, +0.222]	+0.135 [+0.002, +0.268]	+0.109 [+0.023, +0.194]
0.3	+0.274 [+0.156, +0.392]	+0.192 [+0.072, +0.312]	+0.130 [−0.021, +0.281]	+0.157 [+0.063, +0.252]
0.5	+0.172 [+0.093, +0.252]	+0.074 [−0.041, +0.189]	+0.039 [−0.076, +0.154]	+0.142 [+0.002, +0.281]
0.7	+0.197 [+0.086, +0.308]	−0.037 [−0.165, +0.090]	−0.125 [−0.240, −0.011]	−0.090 [−0.196, +0.016]
0.8	+0.124 [+0.040, +0.208]	−0.005 [−0.106, +0.096]	−0.113 [−0.256, +0.029]	−0.148 [−0.234, −0.062]
0.9	+0.180 [+0.074, +0.287]	−0.023 [−0.130, +0.084]	−0.094 [−0.228, +0.040]	−0.104 [−0.198, −0.011]

Continuity with the report split

Rerunning the held-out seeds 20–39 with all ten policies reproduces the original-policy columns of Table 7.1 (identical integer decisions; floating-point differences ≤ 3.3×10^{-16}) and shows the same pattern for the fused variant (Table 10.4).

Table 10.4. Held-out seeds 20–39 (n = 20), pp, mean and [95% CI].

Setting	LAVA − Best-Fit	VM-CA − Best-Fit	Host-CA − Best-Fit	VM-CA-Bayes − Best-Fit	VM-CA-Bayes − LAVA
25%	−0.137 [−0.247, −0.026]	−0.145 [−0.261, −0.030]	−0.115 [−0.234, +0.004]	+0.143 [+0.052, +0.233]	+0.280 [+0.160, +0.400]
50%	+0.069 [−0.022, +0.159]	+0.069 [−0.022, +0.159]	+0.069 [−0.022, +0.159]	+0.298 [+0.187, +0.409]	+0.230 [+0.109, +0.350]
75%	+0.215 [+0.096, +0.333]	+0.215 [+0.096, +0.333]	+0.215 [+0.096, +0.333]	+0.365 [+0.236, +0.494]	+0.150 [+0.009, +0.291]
85%	+0.399 [+0.281, +0.517]	+0.399 [+0.281, +0.517]	+0.399 [+0.281, +0.517]	+0.449 [+0.328, +0.570]	+0.050 [−0.047, +0.147]
100%	+0.598 [+0.490, +0.706]	+0.598 [+0.490, +0.706]	+0.598 [+0.490, +0.706]	+0.675 [+0.546, +0.803]	+0.077 [−0.012, +0.166]
Synthetic	+0.001 [−0.104, +0.105]	+0.114 [+0.007, +0.220]	−0.045 [−0.138, +0.049]	+0.114 [+0.007, +0.220]	+0.113 [−0.010, +0.236]
Oracle	+0.661 [+0.549, +0.774]	+0.661 [+0.549, +0.774]	+0.661 [+0.549, +0.774]	+0.661 [+0.549, +0.774]	identical

11 THREATS TO VALIDITY

- **Calibrated prior.** The fusion prior is the generating profile distribution, the same model the default predictor already uses. In practice it would be estimated from history; Table 10.1 shows the risk of underestimating lifetimes.
- **Known error model.** The scheduler is assumed to know the global accuracy and the error spreads, as the original confidence model also assumes. Both directions of σ misspecification were tested.
- **Error separability.** The σ = 3 incorrect bucket of the protocol spans orders of magnitude and is easy to flag as implausible; milder errors give smaller gains.
- **Metric and workload.** Mixture parameters, profile probabilities and VM shapes are inferred, and only the load was varied. About 68% of the primary metric is the drain tail; the arrival-window metric points the same way for the fused predictor (+0.312, +0.207, +0.076 pp over LAVA at 25%, 50% and 85%). Absolute values are not comparable with production results [1].
- **Reconstruction fidelity.** NILAS outperforms LAVA at most accuracies here (Table 9.2), unlike in [1]; our class, deadline and tie-breaking rules are reconstructed choices. The Oracle is a reference, not an upper bound: NILAS-Bayes at 100% accuracy (+0.599 pp) exceeds the NILAS Oracle (+0.581 pp).
- **Statistics and registration.** Only the eight-test primary family is multiplicity-adjusted. The decomposition, the NILAS-Bayes observation and all robustness cells are descriptive. The registration is a self-timestamped local file.
- **Reconstruction boundary.** The original code, traces and random streams were not recovered. Disagreement with the surviving report does not falsify it, because critical parameters are unknown.

Within these limits the internal comparisons are strong. Every policy sees identical traces and noisy values, every

comparison is paired, admission is equal, and the main claims rest on seeds that were unused when the hypotheses were registered.

12 Conclusion and reproducible artifact

The reconstruction restores a runnable study of lifetime-aware VM placement and confirms that prediction quality is a first-order variable: LAVA helps clearly at high accuracy and hurts at low accuracy.

The earlier proposal, host-level confidence-aware routing, does not survive reconstruction. This revision explains why: the confidence signal is inverted under the default predictor and blind under controlled noise, and isolating hosts by mode costs packing.

The diagnosis leads to a different use of uncertainty: put it into the lifetime estimate. Prior-fused prediction combines each noisy prediction with the profile distribution, handles the cap as censoring and conditions on uptime, with every scheduling rule unchanged. In a pre-registered test on 100 fresh seeds it beats LAVA in 3 of 4 tests (6 of 8 overall), and LAVA's low-accuracy penalty disappears. The gain appears to come from the estimate rather than the gate. It persists, sometimes smaller and not always significant, under milder errors, misspecified error spreads, doubled prior means and doubled load, and it can turn negative when the prior underestimates lifetimes. A prior that does not underestimate lifetimes is therefore the main condition for using it.

Running the artifact

Python 3.9 or newer and the standard library run every experiment; only the report and slide builders need third-party packages (reportlab and python-pptx). From the project directory:

```
python3 -m unittest discover -s tests -v
python3 run.py --suite all --workers 8 --output results/my_all
python3 run_improvement.py --suite all --workers 8 --output results/improvement
python3 scripts/run_robust_extra.py --suite control --workers 8 # and --suite load
python3 scripts/verify_reproduction.py
python3 scripts/build_report.py && python3 scripts/build_deck.py
```

Raw per-seed results, paired summaries, manifests with source hashes, the registration file and figures are under results/ and configs/ [9]. IMPROVEMENT.md and REPRODUCTION_DIFFERENCES.md (both in Chinese) record every assumption and its status. The rebuilt source should not be cited as the lost original commit 377cbb1.

References

[1] J. Ling et al. LAVA: Lifetime-Aware VM Allocation with Learned Distributions and Adaptation to Mispredictions. MLSys, 2025. arXiv:2412.09840.

[2] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune and J. Wilkes. Large-scale cluster management at Google with Borg. EuroSys, 2015.

[3] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura and R. Bianchini. Resource Central: Understanding and Predicting Workloads for Improved Resource Management in Large Cloud Platforms. SOSP, 2017.

[4] H. Barbalho et al. Virtual Machine Allocation with Lifetime Predictions. MLSys, 2023.

[5] O. Hadary et al. Protean: VM Allocation Service at Scale. OSDI, 2020.

[6] E. L. Kaplan and P. Meier. Nonparametric Estimation from Incomplete Observations. Journal of the American Statistical Association, 53(282):457–481, 1958.

[7] Surviving course report and presentation, October 2026. Historical metadata and summary results; original code and traces unavailable.

[8] S. Holm. A Simple Sequentially Rejective Multiple Test Procedure. Scandinavian Journal of Statistics, 6(2):65–70, 1979.

[9] Reconstructed artifact: lava_reproduction/ (results/improvement, results/heldout, results/sweep, results/validation, configs/improvement_protocol.json).